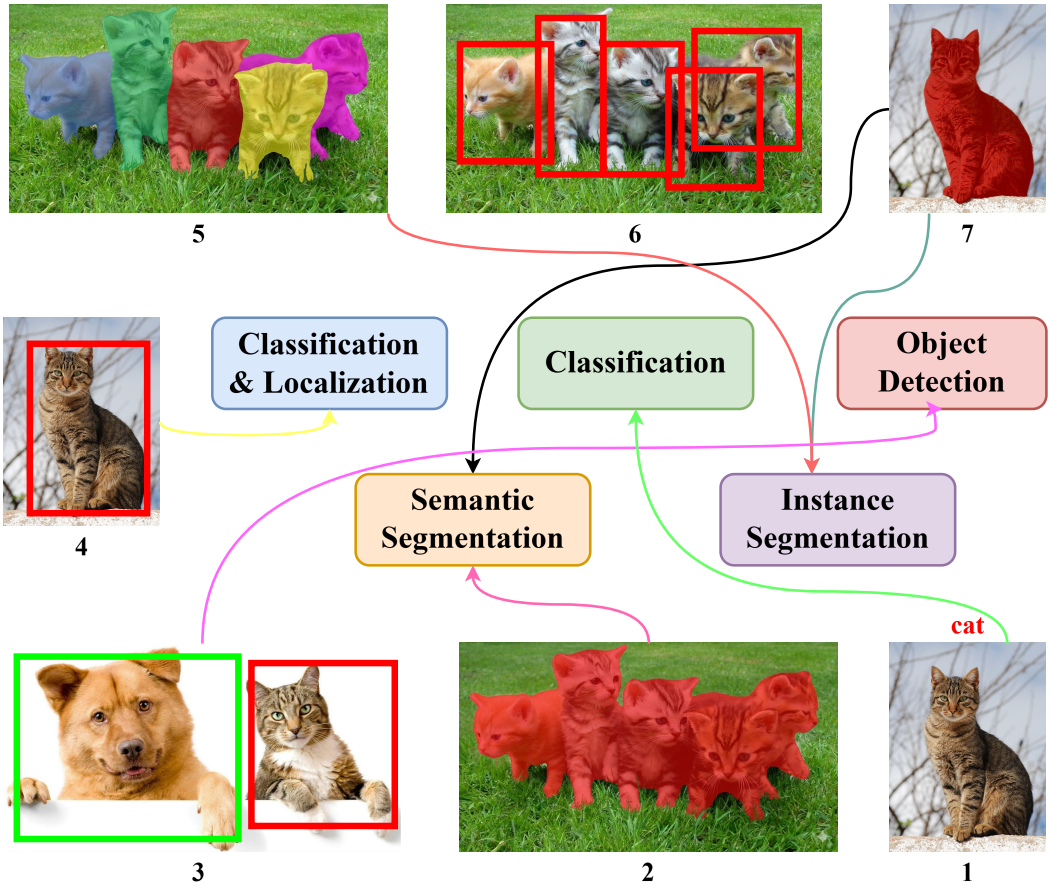
AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

Quiz: Object Detection YOLO

Đậu Văn Nam, Nguyễn Đặng Duy Tân, Nguyễn Phúc Nguyên,
Trần Quang Minh, Nguyễn Khoa và Trần Hoàng Duy

1. Xét các mô hình deep learning khác nhau được huấn luyện cho các bài toán thị giác máy tính riêng biệt. Sơ đồ dưới đây ánh xạ các kết quả đầu ra trực quan (Hình 1–7) với các loại bài toán tương ứng. Biết rằng trong sơ đồ có đúng một ánh xạ vi phạm định nghĩa chuẩn của bài toán (xét theo output representation). Hãy chọn hình đang bị nối sai:



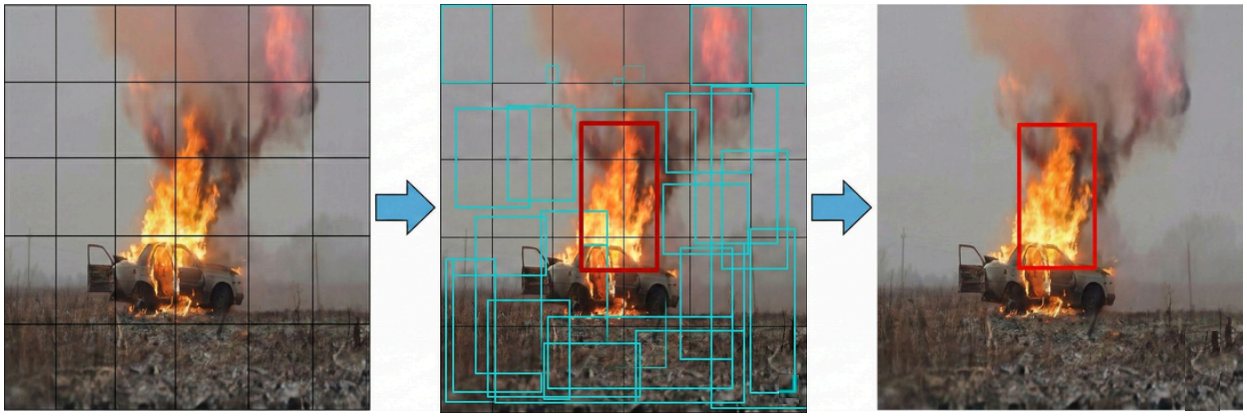
A. Hình (4) → Classification + Localization.

B. Hình (6) → Object Detection.

C. Hình (2) → Semantic Segmentation.

D. Hình (7) → Instance Segmentation.

2. YOLOv1 hoạt động dựa trên nguyên lý chia ảnh đầu vào thành một lưới không gian $S \times S$, giao trách nhiệm dự đoán cho ô lưới chứa tâm của vật thể. Tại mỗi ô lưới, mô hình thực hiện dự báo đồng thời B (bounding boxes) cùng các điểm tin cậy (confidence scores) tương ứng cho từng khung bao đó. Kết quả dự báo sau đó được đưa qua bộ lọc Non-Maximum Suppression (NMS) để loại bỏ các khung bao trùng lặp dựa trên ngưỡng tin cậy và chỉ số IoU. Mặc dù tối ưu hóa tốc độ, nhưng chính cách chia lưới này lại trực tiếp gây ra điểm yếu kỹ thuật lớn nào?



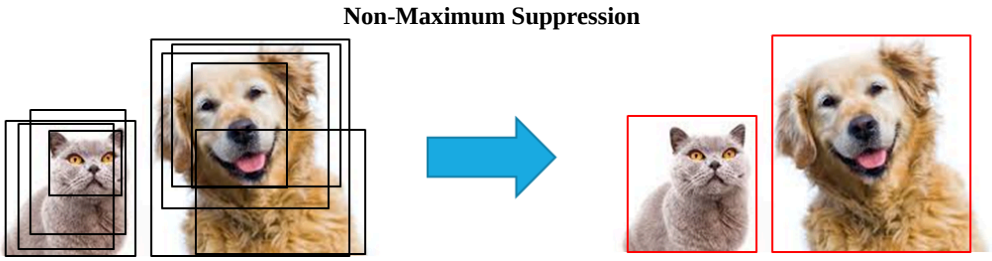
A. Việc chia nhỏ ảnh thành các ô lưới độc lập khiến mô hình chỉ tập trung vào thông tin cục bộ, thiếu các thông tin toàn cục.

B. Cấu trúc lưới cố định khiến cho mô hình không có khả năng đoán được chính xác tọa độ đối với các đối tượng có tỉ lệ khung hình (Aspect Ratio) biến thiên mạnh hoặc kích thước đa dạng.

C. Mô hình không thể nhận diện đồng thời nhiều vật thể khi tâm của chúng cùng hội tụ vào một ô lưới duy nhất, đặc biệt đối với các nhóm vật thể nhỏ đứng sát nhau.

D. Việc chia lưới khiến số lượng dự đoán tỉ lệ thuận với độ phân giải ảnh; do đó, ảnh độ phân giải cao sẽ làm bùng nổ tham số và gây quá tải VRAM GPU.

3. Non-Maximum Suppression là bước hậu xử lý trong bài toán Object Detection nhằm loại bỏ các dự đoán trùng lặp trên cùng một vật thể. Thuật toán hoạt động bằng cách giữ lại bounding box có giá trị confidence cao nhất và triệt tiêu các bounding box có độ chồng lấp IoU vượt qua một ngưỡng định trước. Trong đó cơ chế class-aware NMS đảm bảo quá trình so sánh và triệt tiêu chỉ diễn ra giữa các bounding box thuộc cùng một lớp đối tượng.



Bạn hãy truy cập file notebook NMS from scratch tại Link, hoàn thiện các đoạn code về class-aware NMS (`non_max_suppression`) với ngưỡng confidence và IoU bằng 0.5. Sau đó bạn hãy chạy `non_max_suppression` với các predictions theo YOLO format được cho sẵn, quan sát xem sau khi predictions trải qua NMS thì còn lại bao nhiêu bounding boxes?

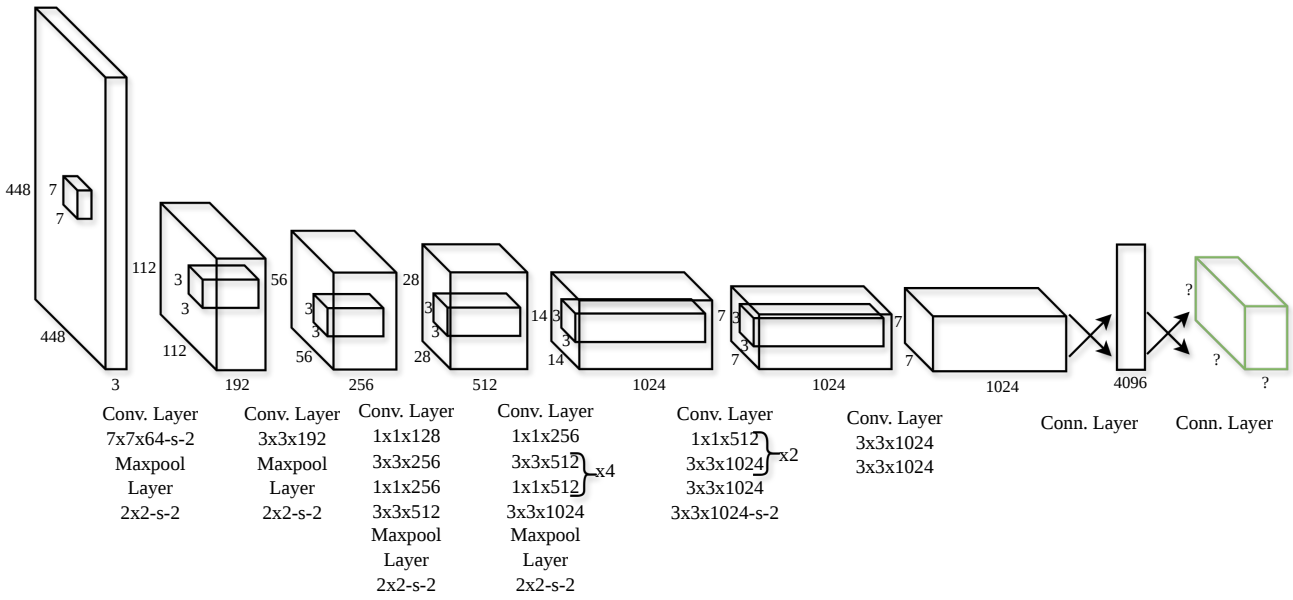
A. 5 bounding boxes.

B. 6 bounding boxes.

C. 7 bounding boxes.

D. 8 bounding boxes.

4. Cho mô hình YOLOv1 nhận ảnh đầu vào kích thước $448 \times 448 \times 3$. Ảnh được chia thành lưới $S = 7$, mỗi ô dự đoán $B = 2$ bounding boxes và $C = 20$ lớp. Tensor đầu ra cuối cùng của mô hình có kích thước bao nhiêu?



A. $(7, 7, 30)$.

B. $(7, 7, 25)$.

C. $(14, 14, 30)$.

D. $(448, 448, 30)$.

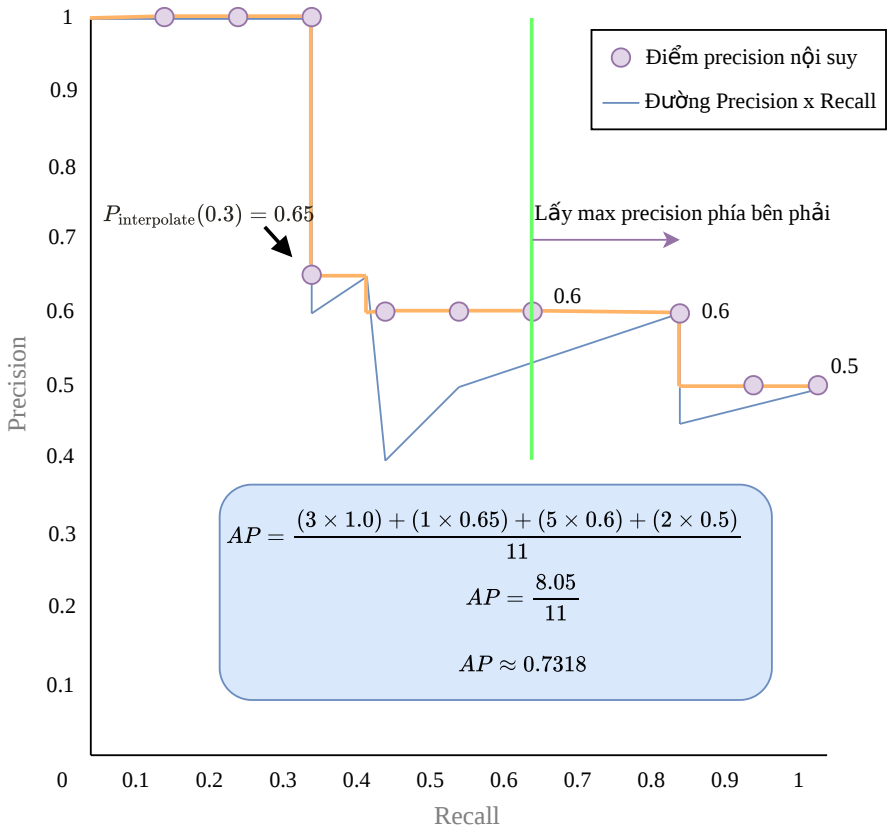
5. Việc đánh giá bài toán Object Detection đòi hỏi mô hình cần tìm kiếm vị trí và phân loại từng object thông qua các Bounding Box. Quá trình này gắn liền với khái niệm Confidence, thể hiện mức độ tự tin đối với mỗi dự đoán được đưa ra. Average Precision (AP) đo lường khả năng tìm kiếm vị trí và phân loại một object thông qua Precision và Recall. Bằng cách sắp xếp dữ liệu theo Confidence giảm dần, AP đánh giá mức độ ổn định của mô hình tại nhiều ngưỡng Confidence khác nhau. Để đánh giá tổng quát hơn trên nhiều class, ta có Mean Average Precision (mAP) được tính thông qua giá trị trung bình AP trên toàn bộ Class (C) theo công thức sau:

$$\text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_{11}^{(c)} \tag{1}$$

Theo chuẩn VOC 2007, AP được định nghĩa là giá trị Precision trung bình tại 11 mức Recall $r \in \{0.0, 0.1, 0.2, \ldots, 1.0\}$. Tại mỗi r, chúng ta không lấy giá trị Precision thực tế tại điểm đó mà lấy giá trị Precision lớn nhất ứng với bất kỳ mức Recall nào lớn hơn hoặc bằng r:

$$p_{\text{interpolate}}(r) = \max_{\tilde{r} \geq r} p(\tilde{r}) \tag{2}$$

Nói cách khác, giá trị nội suy tại một điểm chính là "đỉnh cao nhất" tính từ điểm đó đổ về phía bên phải đồ thị (Bạn tham khảo ví dụ tại Hình 1).



Hình 1: Minh họa phương pháp nội suy 11 điểm theo chuẩn VOC 2007

Sau khi đã xác định được 11 giá trị Precision nội suy (p_{interp}), giá trị AP cuối cùng là trung bình cộng của chúng:

$$\text{AP}_{11} = \frac{1}{11} \sum_{r \in \{0.0, 0.1, \ldots, 1.0\}} p_{\text{interp}}(r) \tag{3}$$

Giả sử chúng ta đang đánh giá một mô hình object detection. Qua quá trình đối chiếu với Ground Truth (GT) và Non-Maximum Suppression, chúng ta thu được 7 dự đoán của một class với Confidence như sau:

Dự đoán	**Confidence**	**Kết quả**
Sample 1	0.63	TP
Sample 2	0.77	TP
Sample 3	0.92	TP
Sample 4	0.86	FP
Sample 5	0.88	TP
Sample 6	0.58	TP
Sample 7	0.91	FP

Kết quả Average Precision (AP) của class này là:

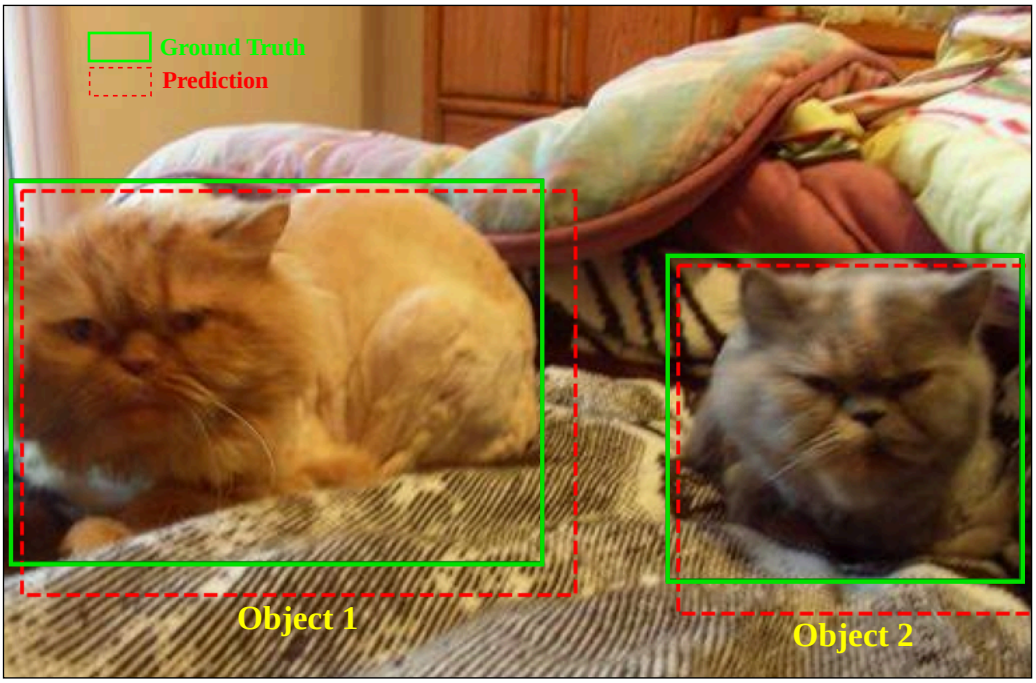
A. 0.349.

B. 0545.

C. 0.5.

D. 0.789.

6. Trong kiến trúc YOLOv1, khi tính toán Loss cho phần toạ độ bounding box với các kích thước khác nhau, tác giả tính sai số dựa trên $\sqrt{w}$ và $\sqrt{h}$ thay vì tính trực tiếp trên w, h.



Kích thước Bounding Boxes

- **Object 1 (Object có kích thước bounding box lớn hơn)**

$$\text{GT (Pixels)} : (x_{\min}, y_{\min}, w, h) = (4.0,\ 80.5,\ 246.0,\ 177.3)$$
$$\text{Pred (Pixels)} : (x_{\min}, y_{\min}, w, h) = (9.0,\ 85.5,\ 256.0,\ 187.3)$$

- **Object 2 (Object có kích thước bounding box nhỏ hơn)**

$$\text{GT (Pixels)} : (x_{\min}, y_{\min}, w, h) = (307.9,\ 114.9,\ 163.7,\ 151.2)$$
$$\text{Pred (Pixels)} : (x_{\min}, y_{\min}, w, h) = (312.9,\ 119.9,\ 173.7,\ 161.2)$$

Bounding Box Regression Loss (YOLOv1):

$$\mathcal{L}_{\text{box}} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{obj} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

Hỏi tính sai số trên $\sqrt{w}$ và $\sqrt{h}$ giúp giải quyết vấn đề gì trong quá trình huấn luyện mô hình?

A. Để tránh việc giá trị loss trở thành số âm trong quá trình hội tụ, giúp đạo hàm ổn định hơn.

B. Để loại trừ nhiễu từ các phông nền (background) lớn không chứa đối tượng nào.

C. Để tăng mức phạt cho sai lệch ở bounding box nhỏ hơn, đồng thời giảm mức phạt cho cùng một sai lệch ở bounding box lớn hơn.

D. Để đảm bảo độ đo mAP tăng nhanh hơn ở giai đoạn cuối quá trình huấn luyện bằng cách nén không gian toạ độ lại.

7. Trong Notebook Colab đã được cung cấp (Link Colab), lớp `YoloLoss` được cài đặt với hai hệ số điều chỉnh mức độ đóng góp của từng thành phần Loss như sau:

Hệ số điều chỉnh

```
# These are YOLO-specific constants, representing the weight
# for no object loss (lambda_noobj) and box coordinates loss (lambda_coord).
self.lambda_noobj = 0.5
self.lambda_coord = 5
```

Hai hệ số này điều chỉnh mức độ đóng góp của các thành phần khác nhau trong hàm mất mát, bao gồm No-Object Loss (liên quan đến các ô lưới không chứa đối tượng) và Bounding Box Coordinate Loss (liên quan đến sai lệch tọa độ của bounding box dự đoán).

Hãy tạo một bản copy Notebook và chỉnh sửa lại hai tham số trên thành:

Hệ số điều chỉnh

```
self.lambda_noobj = 10.0
self.lambda_coord = 0.1
```

Lưu ý: Giữ nguyên toàn bộ cấu hình huấn luyện (learning rate, optimizer, batch size, số epochs, seed, dataset, ...). Chỉ thay đổi hai hệ số λ_{noobj} và λ_{coord} như yêu cầu.

Sau khi huấn luyện lại mô hình, hãy đối chiếu các chỉ số Train Loss, Validation Loss và mAP với Notebook ban đầu. Hỏi sự thay đổi nào dưới đây mô tả đúng nhất tác động của việc tăng mạnh λ_{noobj} và giảm mạnh λ_{coord} đến quá trình huấn luyện và hiệu năng phát hiện đối tượng của mô hình?

A. Mô hình hội tụ nhanh hơn và đạt mAP cao hơn vì việc tăng mạnh λ_{noobj} giúp loại bỏ ảnh hưởng của nền (background), làm quá trình học tập trung hơn vào vật thể.

B. mAP duy trì ở mức rất thấp trong nhiều epoch đầu và giá trị mAP tốt nhất cuối cùng suy giảm đáng kể.

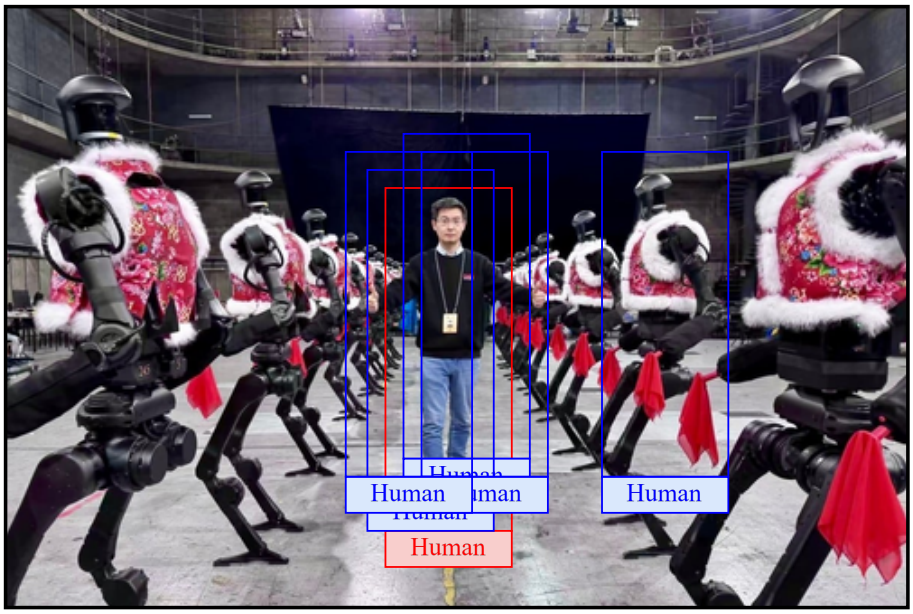
C. Loss trở nên không ổn định và xuất hiện hiện tượng exploding gradients do λ_{noobj} quá lớn.

D. Không có thay đổi đáng kể về mAP vì các thành phần trong Loss tự động cân bằng thông qua cơ chế chuẩn hóa nội tại.

8. Việc sử dụng NMS để lọc bounding box trong các bản YOLO trước (v1-v8) làm tăng độ trễ và cản trở khả năng tối ưu hóa end-to-end của mô hình. Để loại bỏ NMS, YOLOv10 đề xuất Consistent Dual Label Assignment, chạy song song hai nhánh dự đoán trong quá trình huấn luyện:

   - Nhánh One-to-Many (o2m): Gán một ground-truth cho nhiều dự đoán (Top-k) nhằm cung cấp tín hiệu giám sát phong phú, giúp mạng tối ưu trong việc trích xuất đặc trưng.
   - Nhánh One-to-One (o2o): Chỉ gán ground-truth cho một dự đoán tốt nhất (Top-1). Khi suy luận thực tế (inference), mô hình chỉ sử dụng duy nhất nhánh o2o này để xuất ra kết quả, qua đó loại bỏ hoàn toàn sự phụ thuộc vào NMS.

   Xét một ảnh có 1 ground-truth box b và mô hình dự đoán ra 5 dự đoán với các giá trị được minh họa trong ảnh dưới đây:



Bảng kết quả dự đoán của mô hình

i	s_i	p_i	IoU_i
1	1	0.35	0.65
2	1	0.70	0.40
3	1	0.55	0.58
4	0	0.95	0.92
5	1	0.45	0.75

- Dự đoán i: Kết quả dự đoán của mô hình.
- s_i (Spatial Prior): Ràng buộc vùng không gian.
- p_i (Classification Score): Điểm phân loại.
- $IoU_i = IoU(\hat{b}_i, b)$: Tỷ lệ giữa diện tích phần giao và phần hợp của dự đoán ($\hat{b}_i$) và GT (b).

Công thức Matching metric dùng cho cả hai nhánh:

$$m(\alpha, \beta) = s \cdot p^{\alpha} \cdot IoU^{\beta}.$$

Với nhánh one-to-many (o2m): $\alpha_{o2m} = 0.5,\ \beta_{o2m} = 6$.

Nhánh one-to-one (o2o): lấy $r = 1$ nên $\alpha_{o2o} = \alpha_{o2m},\ \beta_{o2o} = \beta_{o2m}$.

Quy tắc gán:

- o2m chọn Top-3 theo m làm tập positive Ω.
- o2o chọn Top-1 theo m làm match duy nhất.

Để tính hàm Loss phân loại cho nhánh o2m, nhãn mềm (soft label) được chuẩn hóa theo mẫu tốt nhất trong tập Ω. Gọi u^* là giá trị IoU lớn nhất trong Ω và m^*_{o2m} là điểm matching lớn nhất. Nhãn mềm $t_{o2m,k}$ cho dự đoán $k \in \Omega$ được tính bằng:

$$t_{o2m,k} = u^* \cdot \frac{m_{o2m,k}}{m^*_{o2m}}$$

Dựa vào các thông số trên, giá trị nhãn mềm của dự đoán số 1 ($t_{o2m,1}$) bằng bao nhiêu? (Các phép tính làm tròn 4 chữ số sau dấu phẩy).

A. 0.2801.

B. 0.7500.

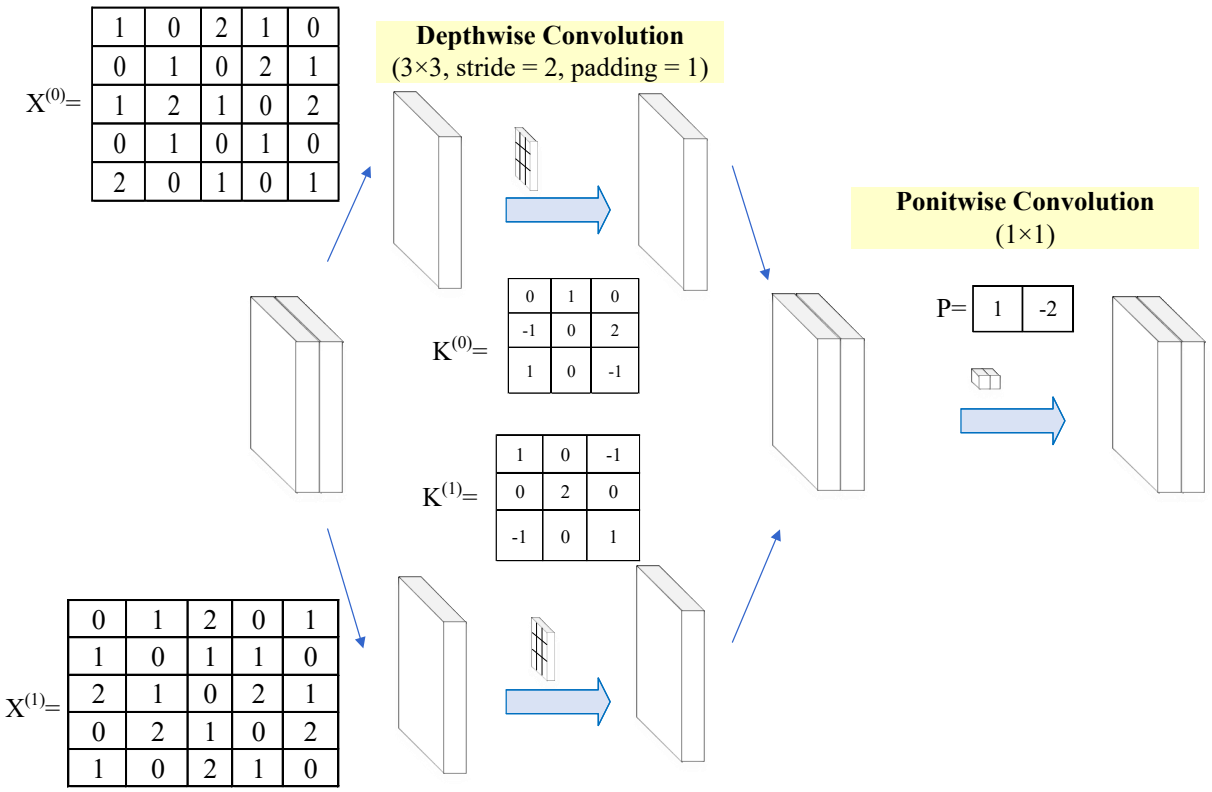
C. 0.0446.

D. 0.1772.

9. Trong YOLOv10, nhóm tác giả thay thế phép convolution thông thường bằng tổ hợp **Depthwise Convolution + Pointwise Convolution** nhằm giảm số lượng tham số và chi phí tính toán:

   - Lightweight classification head sử dụng **depth-wise conv** (3×3) kết hợp **point-wise conv** (1×1).
   - Spatial-channel decoupled downsampling phân tách phép conv thành point-wise (điều chỉnh số channel) và depth-wise (điều chỉnh spatial).

   Xét block sau:



Depthwise Conv (3×3, stride = 2, padding = 1) $\rightarrow$ **Pointwise Conv** (1×1, 1 output)

(Không bias, không activation)

- **Input** $(5 \times 5,\ 2$ channels):

$$X^{(0)} = \begin{bmatrix} 1 & 0 & 2 & 1 & 0 \\ 0 & 1 & 0 & 2 & 1 \\ 1 & 2 & 1 & 0 & 2 \\ 0 & 1 & 0 & 1 & 0 \\ 2 & 0 & 1 & 0 & 1 \end{bmatrix} \qquad X^{(1)} = \begin{bmatrix} 0 & 1 & 2 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 2 & 1 & 0 & 2 & 1 \\ 0 & 2 & 1 & 0 & 2 \\ 1 & 0 & 2 & 1 & 0 \end{bmatrix}$$

- **Depthwise kernels:**

$$K^{(0)} = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 2 \\ 1 & 0 & -1 \end{bmatrix} \qquad K^{(1)} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 2 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

- **Pointwise filter:**

$$P = [\,1,\ -2\,]$$

Giá trị tại vị trí $(1, 1)$ của output cuối cùng bằng bao nhiêu?

A. -2

B. 0

C. 4

D. 6

10. Trong lĩnh vực bán lẻ và logistics, công nghệ Object Detection (Nhận diện vật thể) đóng vai trò quan trọng trong việc tối ưu hóa quy trình kiểm kê và quản lý tồn kho. Hệ thống nhận diện tự động không chỉ hỗ trợ doanh nghiệp theo dõi số lượng hàng hóa mà còn cung cấp dữ liệu định lượng để đánh giá hiệu quả trưng bày thương hiệu tại điểm bán dựa trên mật độ bao phủ của các loại nhãn hàng.

    Hãy sử dụng thư viện Ultralytics để huấn luyện mô hình YOLOv10n nhận diện các đối tượng mục tiêu trong tập dữ liệu sau (link). Tập dữ liệu bao gồm 19 loại đối tượng mục tiêu, tiêu biểu như: các loại lon bia, thùng bia, biển quảng cáo, nhân sự marketing, ... Tuy nhiên, trong phạm vi bài tập này, chúng ta chỉ tập trung vào đối tượng **lon bia** nhằm thực hành kỹ năng theo dõi số lượng và phân loại vật thể cụ thể trong một tập dữ liệu đa mục tiêu.

    Các thông số cấu hình cho giai đoạn huấn luyện và dự đoán được thiết lập thống nhất như sau:

Tham số	**Giá trị**
Giai đoạn Huấn luyện (sử dụng trong hàm train)	
Seed	42
Epochs	100
Batch size	32
Giai đoạn Dự đoán (sử dụng trong hàm predict)	
Confidence Threshold	0.2
IoU Threshold	0.7
* Các thông số khác giữ nguyên mặc định	

    Sau khi hoàn tất quá trình huấn luyện, sử dụng trọng số tốt nhất (best checkpoint) để tiến hành nhận diện trên tệp hình ảnh count_beers.jpg.



Hình 2: Ảnh cần thực hiện đếm số lượng lon bia

Dựa trên kết quả trả về từ mô hình, hãy thống kê tổng số lượng **lon bia mà mô hình nhận diện được** (lưu ý: loại trừ các đối tượng là thùng bia).

A. 256.

B. 187.

C. 89.

D. 223.

AI VIET NAM
@aivietnam.edu.vn

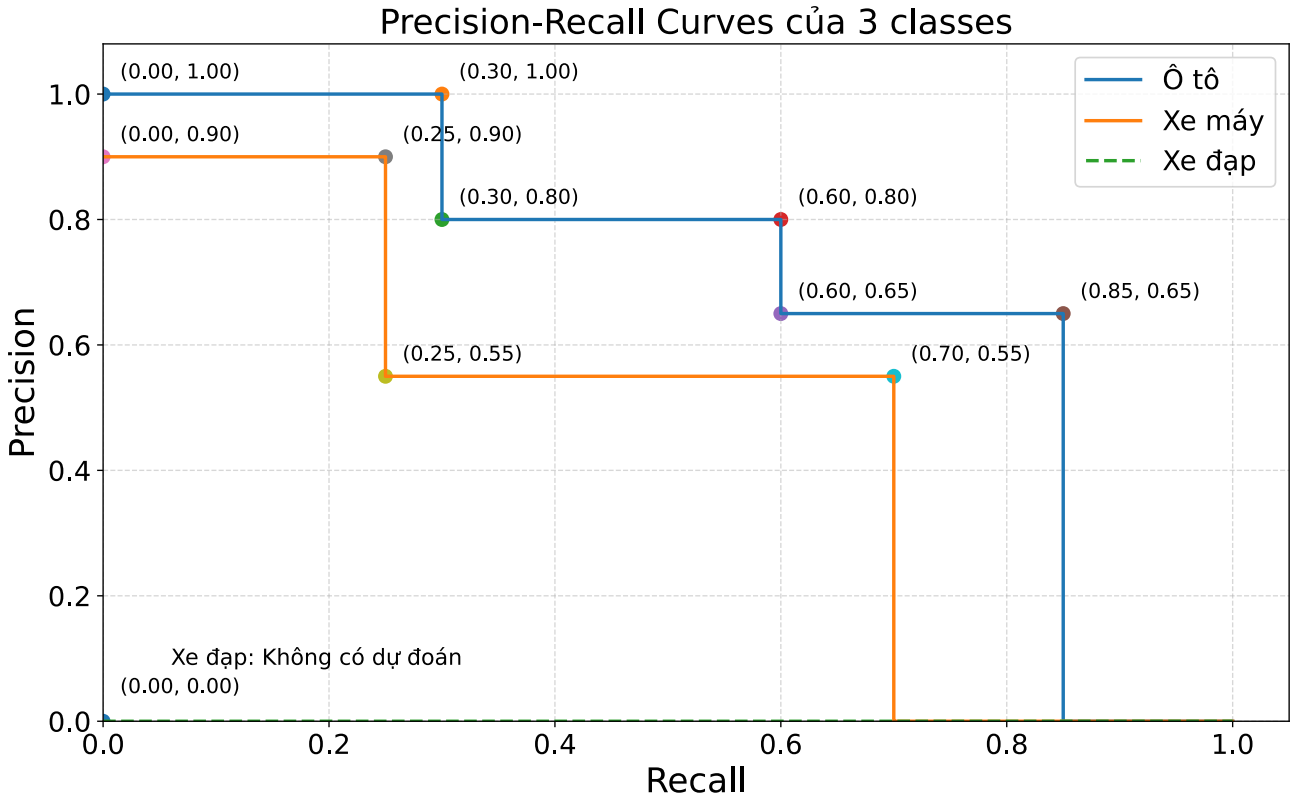
AI VIET NAM COURSE 2025

Quiz: Object Detection

Nguyễn Thế Hào, Huỳnh Khôi Minh Uyên, Hồ Đăng Mạnh, Chu Tuấn Nghĩa,
Lê Nguyễn Phước Thành và Phạm Quang Vinh

I. Đề bài

Câu 1: Giả sử một mô hình phát hiện đối tượng kiểu YOLOv1 được huấn luyện để nhận diện 3 lớp: Ô tô, Xe máy, và Xe đạp. Sau khi chạy suy luận trên tập kiểm tra, ta thu được các dự đoán kèm confidence score và từ đó ta có được Precision-Recall Curve cho từng class như hình dưới đây.



Biết rằng:

- **AP (Average Precision)** của một class là diện tích dưới đường Precision-Recall Curve của class đó.
- **mAP (mean Average Precision)** là trung bình cộng các giá trị AP của tất cả các class.

Dựa trên các đường PR trong hình, giá trị **mAP** của mô hình là:

(a) 0.29

(b) 0.39

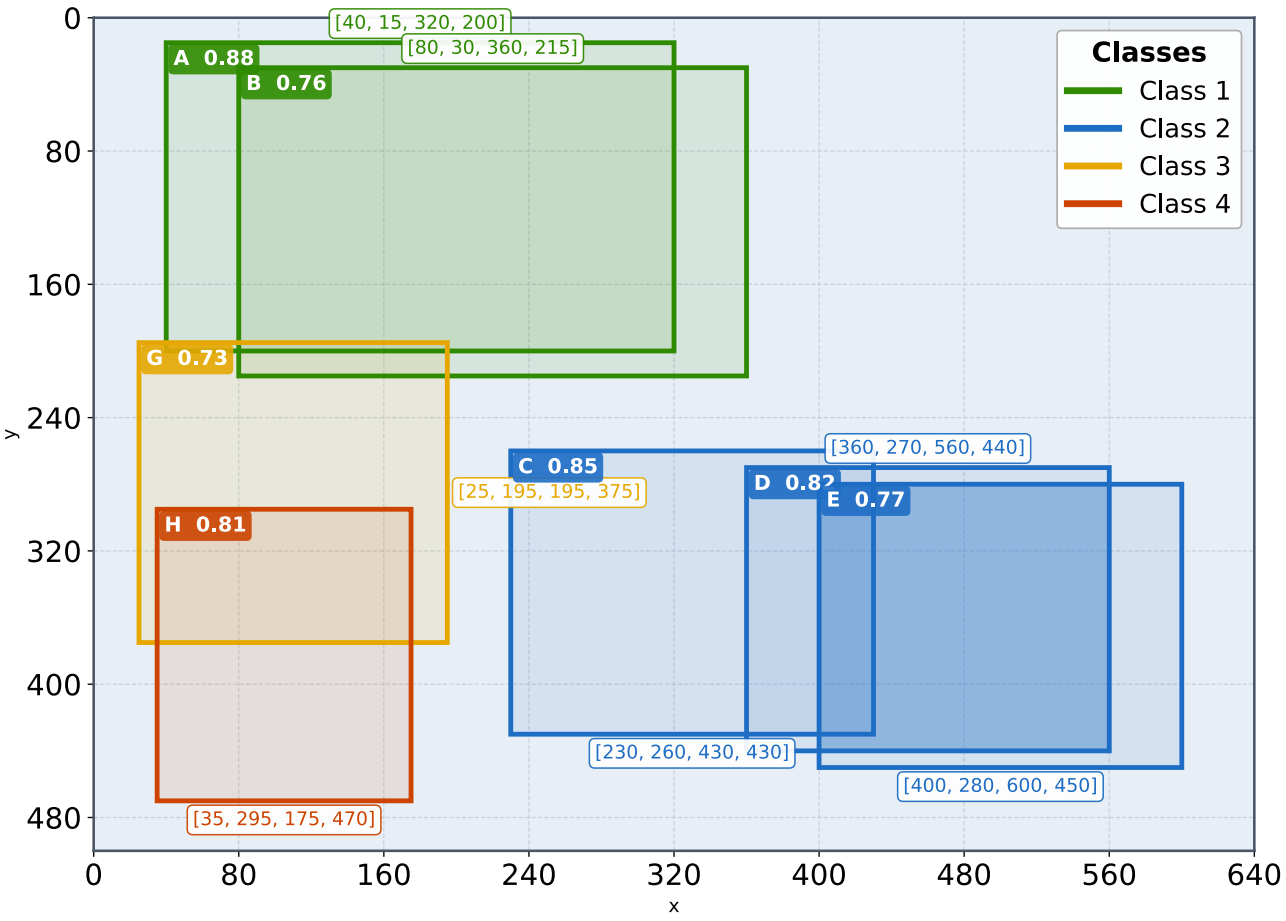
(c) 0.49

(d) 0.59

Câu 2: Áp dụng thuật toán Non-Maximum Suppression (NMS) cho các predicted bounding boxes trong hình dưới đây. Tọa độ mỗi bounding box được biểu diễn theo định dạng $[x_{min}, y_{min}, x_{max}, y_{max}]$.

- IoU threshold = 0.5
- NMS được áp dụng độc lập cho từng class
- Trong mỗi class, các bounding boxes được xét theo thứ tự confidence score giảm dần

Sau khi thực hiện NMS, những bounding boxes nào còn lại?



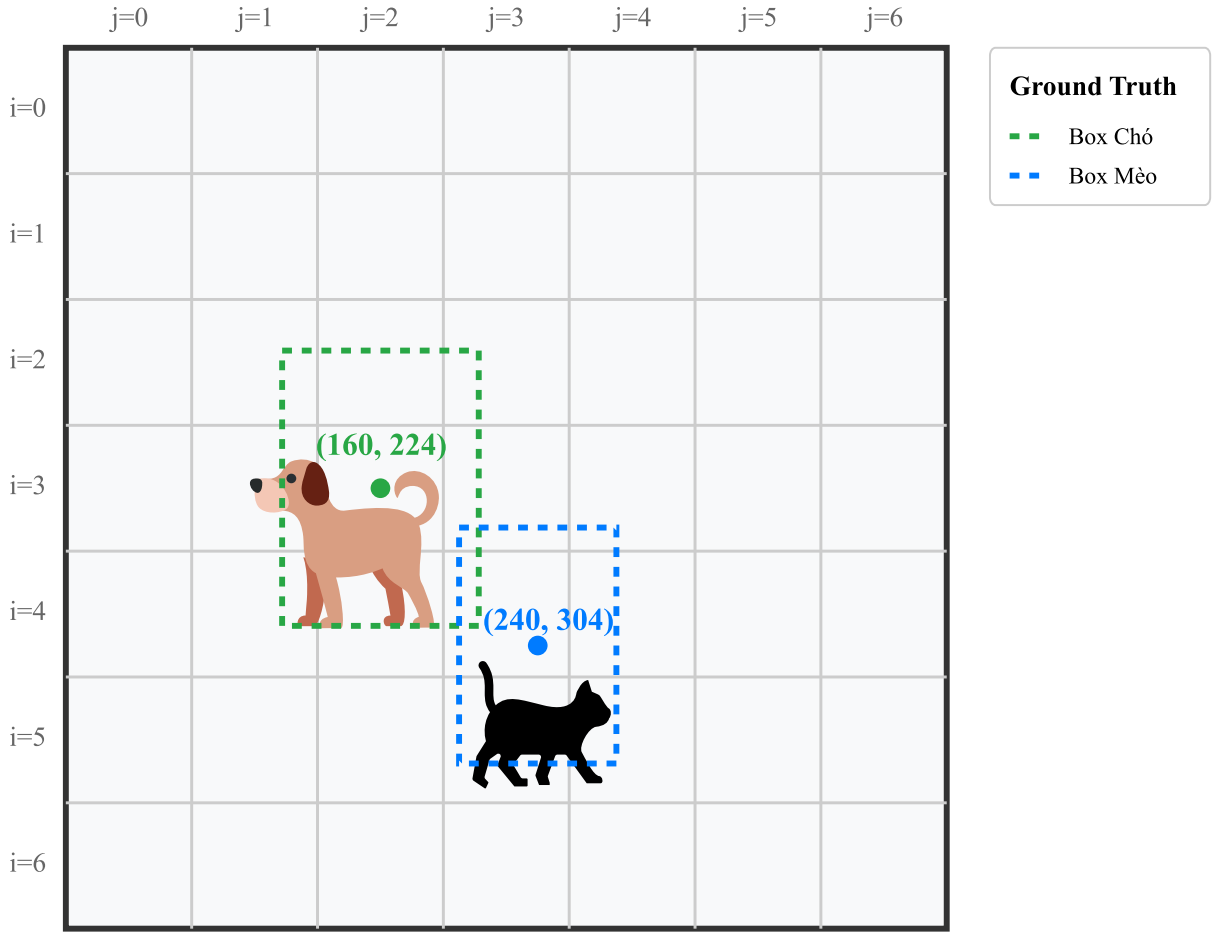
(a) A 0.88, G 0.73, H 0.81, C 0.85

(b) A 0.88, B 0.76, G 0.73, H 0.81, C 0.85

(c) A 0.88, G 0.73, H 0.81, D 0.82, C 0.85

(d) A 0.88, G 0.73, H 0.81, E 0.77

Câu 3: Xét mạng YOLOv1 xử lý ảnh đầu vào có kích thước $(W, H) = (448, 448)$ và chia thành lưới $S \times S$ (với $S = 7$). Tọa độ tâm (center) dạng pixel của hai vật thể Ground Truth (**Chó** và **Mèo**) được cung cấp chi tiết như hình.



Dựa vào công thức chuẩn hóa tọa độ của YOLOv1:

$$x_{\text{norm}} = \frac{x}{W}, \quad j = \lfloor S \cdot x_{\text{norm}} \rfloor, \quad x_{\text{cell}} = S \cdot x_{\text{norm}} - j$$

(áp dụng tương tự cho trục y để tìm i và y_{cell}). Hãy xác định cell (i, j) (0-index) chịu trách nhiệm nhận diện cho từng vật thể và tọa độ tâm chuẩn hóa theo cell tương ứng $(x_{\text{cell}}, y_{\text{cell}})$:

(a) Chó thuộc Cell $(2, 3)$ tâm $(0.5, 0.5)$; Mèo thuộc Cell $(3, 4)$ tâm $(0.75, 0.75)$

(b) Chó thuộc Cell $(3, 2)$ tâm $(2.5, 3.5)$; Mèo thuộc Cell $(4, 3)$ tâm $(3.75, 4.75)$

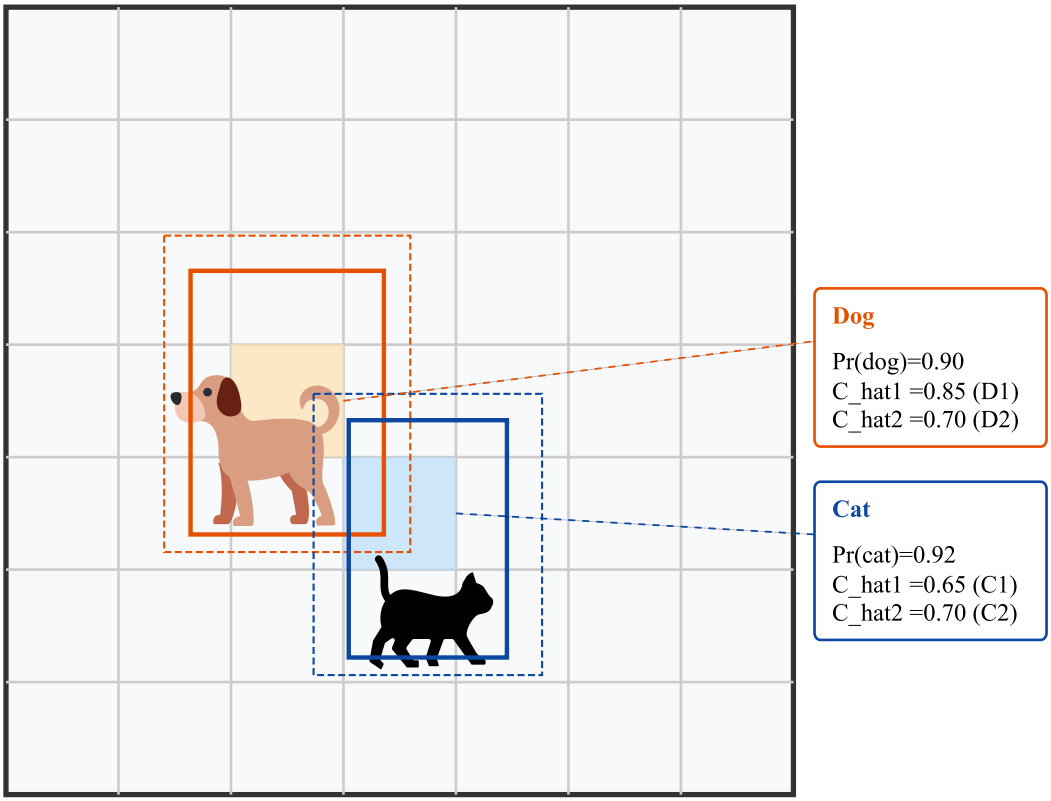
(c) Chó thuộc Cell $(3, 2)$ tâm $(0.5, 0.5)$; Mèo thuộc Cell $(4, 3)$ tâm $(0.75, 0.75)$

(d) Chó thuộc Cell $(4, 3)$ tâm $(0.5, 0.5)$; Mèo thuộc Cell $(3, 2)$ tâm $(0.75, 0.75)$

Câu 4: Sử dụng kết quả định vị Cell từ Câu 3, mạng YOLOv1 tiến hành Pha Suy luận (Inference) tại hai cell này. Trong kiến trúc YOLOv1, mỗi cell sẽ dự đoán:

- **1 vector xác suất lớp** có điều kiện $\Pr(\text{Class}_k \mid \text{Object})$ (dùng chung cho cả B boxes trong cell).
- $B = 2$ **boxes**, mỗi box có Objectness Confidence riêng $\hat{C}_b$.

Dữ liệu mạng xuất ra tại **Cell chứa Chó** và **Cell chứa Mèo** được tóm tắt trong **Hình 2**.



Với ngưỡng lọc sơ bộ `SCORE_THRESHOLD` $= 0.55$, ta tính điểm tự tin phân lớp bằng công thức:

$$\text{Score}_{b,k} = \Pr(\text{Class}_k \mid \text{Object}) \times \hat{C}_b.$$

Ở bước **lọc theo Score (pre-NMS)**, hệ thống **giữ tất cả** các bounding boxes có Score $\geq$ `SCORE_THRESHOLD`.
Hãy chọn **đúng** tập ứng viên (các box được giữ lại) và score tương ứng:

(a) Giữ {D1} với 0.765; giữ {C1,C2} với $(0.598, 0.644)$.

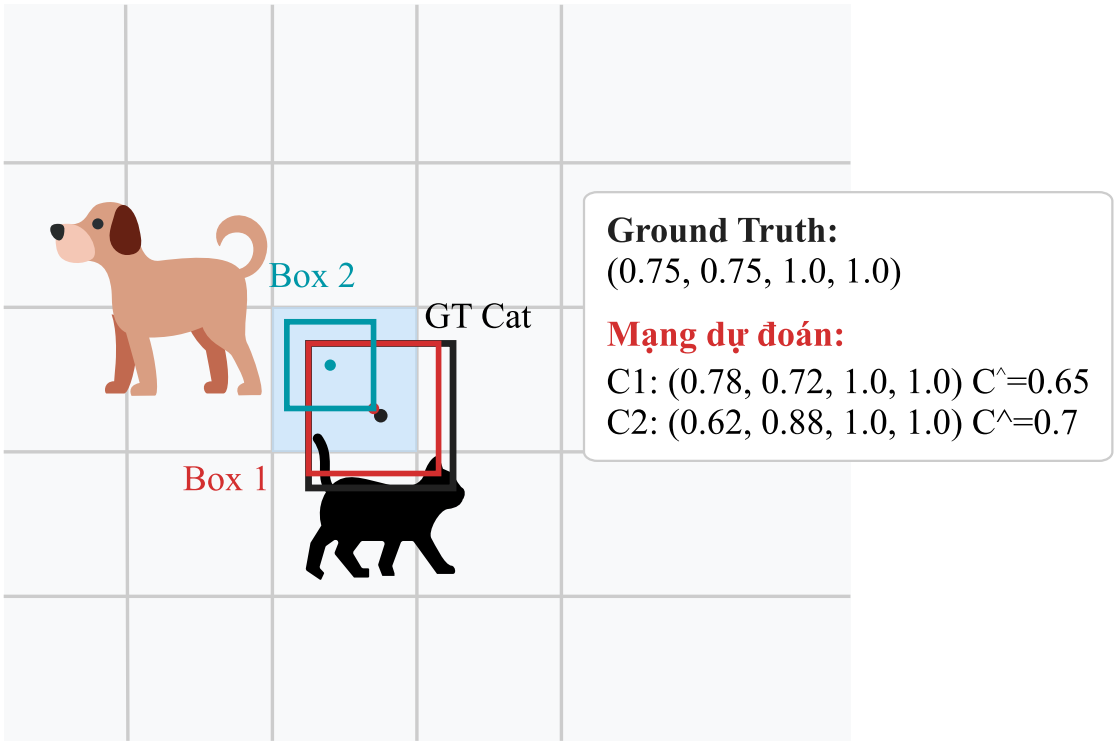
(b) Giữ {D1,D2} với $(0.765, 0.630)$; giữ {C2} với 0.644.

(c) Giữ {D1,D2} với $(0.765, 0.630)$; giữ {C1,C2} với $(0.598, 0.644)$.

(d) Giữ {D2} với 0.630; giữ {C1} với 0.598.

Câu 5: Quay lại pha **Training** tại **Cell chịu trách nhiệm cho Mèo**. Sử dụng tọa độ tâm GT $(x_{\text{cell}}, y_{\text{cell}})$ đã tìm được ở **Câu 3**, kết hợp với kích thước thực tế $w_{\text{cell}} = 1.0$, $h_{\text{cell}} = 1.0$. Mạng dự đoán đúng **2 boxes** tại cell này tương tự Câu 4 (mô tả trực quan tại):

- **Box C1:** $(\hat{x}, \hat{y}, \hat{w}, \hat{h}) = (0.78,\ 0.72,\ 1.00,\ 1.00), \quad \hat{C}_1 = 0.65.$
- **Box C2:** $(\hat{x}, \hat{y}, \hat{w}, \hat{h}) = (0.62,\ 0.88,\ 1.00,\ 1.00), \quad \hat{C}_2 = 0.70.$



Box **responsible** là box có IoU với GT lớn hơn. Cho các hệ số phạt $\lambda_{\text{coord}} = 5$ và $\lambda_{\text{noobj}} = 0.5$. **Giả sử** target confidence/objectness của responsible box là 1, và box còn lại trong cell là **no-object** (target = 0). Xét **loss tại cell này** chỉ gồm:

$$\text{Loss}_{xy} = \lambda_{\text{coord}} \left[(x - \hat{x})^2 + (y - \hat{y})^2 \right],$$

$$\text{Loss}_{conf} = (1 - \hat{C}_{resp})^2 + \lambda_{\text{noobj}} (0 - \hat{C}_{other})^2.$$

(*Bỏ qua classification loss và loss của các cell khác*). Tổng loss tại cell này $(\text{Loss}_{xy} + \text{Loss}_{conf})$ gần đúng bằng:

(a) 0.0715

(b) 0.3765

(c) 0.3075

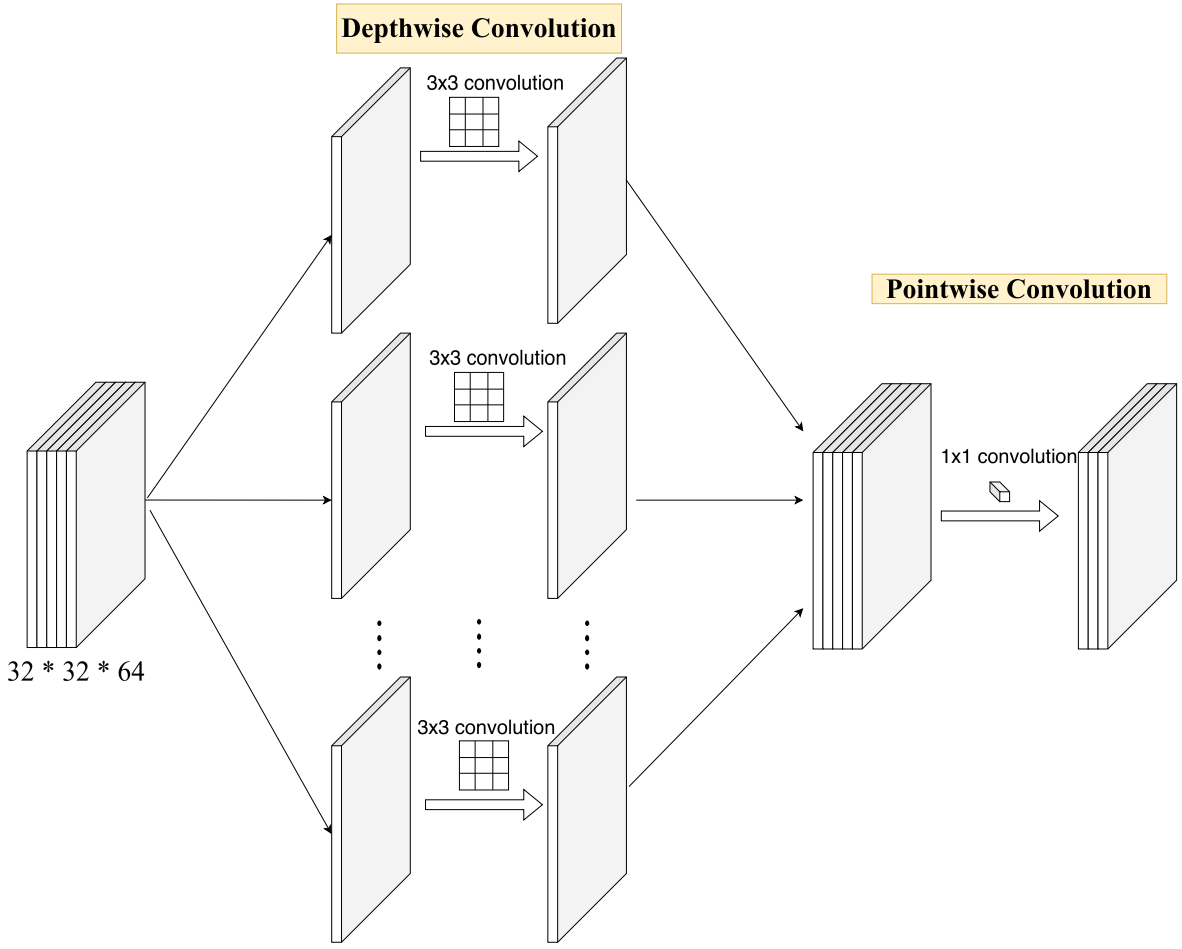
(d) 0.5625

Câu 6: Cho input feature map kích thước $32 \times 32 \times 64$. Áp dụng depthwise separable convolution gồm:

- DWConv 3×3 (depthwise multiplier = 1, stride=1, padding "same"),
- sau đó PWConv 1×1 để giảm kênh $64 \rightarrow 16$,

bỏ qua bias.

Hỏi: (i) Kích thước đầu ra cuối cùng là gì, và (ii) tổng số phép nhân–cộng xấp xỉ bằng bao nhiêu?



(*a*) Output $32 \times 32 \times 16$; Ops = 1,638,400

(*b*) Output $32 \times 32 \times 64$; Ops = 1,638,400

(*c*) Output $32 \times 32 \times 16$; Ops = 1,048,576

(*d*) Output $32 \times 32 \times 16$; Ops = 589,824

Câu 7: Anchor-based YOLO (YOLOv2 trở đi) bổ sung **anchor boxes** (tức là các bounding box mẫu có kích thước cố định). Thay vì dự đoán trực tiếp (w, h), model dự đoán độ lệch (offset) so với ancho. Kĩ thuật này đã thay đổi bài toán từ *predict absolute box* sang *predict offset relative to prior box*.

Trong mỗi grid cell (i, j) và mỗi anchor b, mô hình dự đoán:

$$\big(p_1, p_2, \ldots, p_C,\ t_{obj}, t_x, t_y, t_w, t_h\big)$$

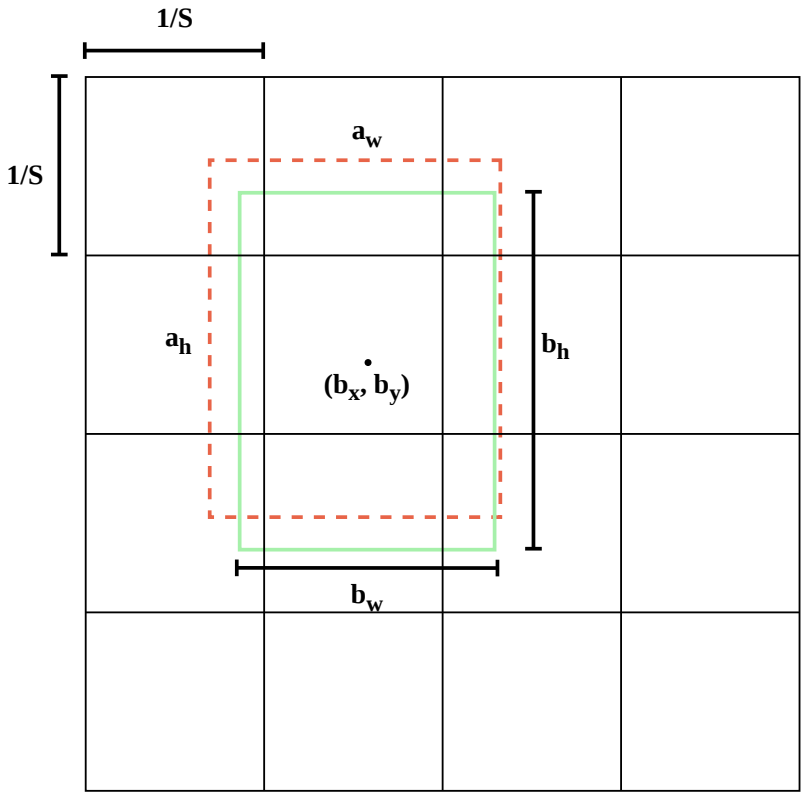
- p_c: xác suất thuộc lớp c
- t_{obj}: objectness score (xác suất có vật thể)
- t_x, t_y: offset tâm box so với góc trái trên của cell
- t_w, t_h: điều chỉnh kích thước dựa trên anchor

Từ Output, ta chuyển đổi sang Bounding Box như sau:

$$b_x = \frac{\sigma(t_x) + j}{S}, \quad b_y = \frac{\sigma(t_y) + i}{S}, \quad b_w = \frac{a_w \cdot e^{t_w}}{S}, \quad b_h = \frac{a_h \cdot e^{t_h}}{S}$$

- (a_w, a_h): kích thước của anchor box b tại cell (i, j)

Bounding box cuối cùng là (b_x, b_y, b_w, b_h), tọa độ được chuẩn hóa về khoảng $[0, 1]$ so với kích thước ảnh.



Dựa trên nội dung mô tả các bạn hãy hoàn thành hàm **forward** của class YoloLoss cho Anchor-based Yolo trên Notebook. Trong đó, các bạn cần hoàn thiện **2 phần TODO** gồm: *Box regression loss* và *Objectness loss*.

- **Box regression loss** (`box_loss`): phạt sai lệch vị trí và kích thước hộp dự đoán (t_x, t_y, t_w, t_h) **chỉ trên những anchor được gán vật thể** (anchor positive).
- **Objectness loss** (`object_loss`): buộc độ tin cậy `objectness` của **anchor positive** tiến gần về 1 (có vật thể).
- **No-object loss** (`no_object_loss`): phạt `objectness` của **anchor negative** (không gán vật thể) nếu dự đoán khác 0.
- **Class loss** (`class_loss`): phạt sai lệch phân loại **chỉ trên các cell có ít nhất một vật thể**.

Sau đấy chạy với testcase bên dưới và cho biết *loss_fn(preds, target)* là bao nhiêu?

```
loss_fn = YoloLoss(S=7, B=2, C=20, anchors=[[1.0, 1.0], [2.0, 1.5]])
preds = torch.randn(1, 7 * 7 * (20 + 5 * 2))
target = torch.zeros(1, 7, 7, 20 + 5 * 2)
target[0, 2, 3, 5] = 1.0
obj_idx = 20 + 0 * 5
target[0, 2, 3, obj_idx] = 1.0
target[0, 2, 3, obj_idx + 1: obj_idx + 5] = torch.tensor([0.4, 0.6, 0.1,
    -0.2])
print("YoloLoss output:", float(loss_fn(preds, target)))
```

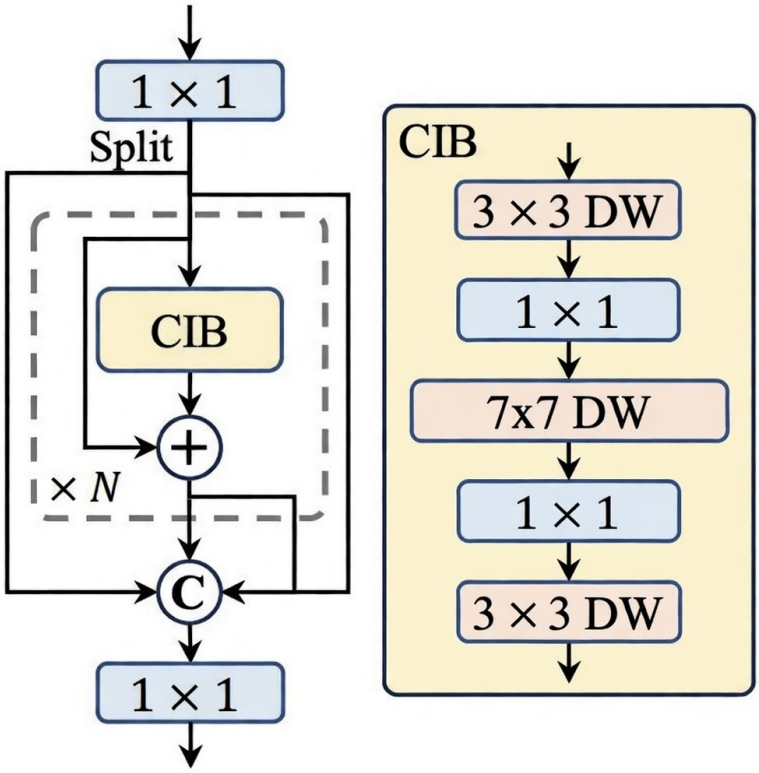
(a) 78.7753

(b) 79.7653

(c) 68.5732

(d) 69.7653

Câu 8: Paper YOLOv10 gốc chọn nâng cấp middle 3×3 DW → 7×7 DW ở các stage sâu hơn của small models. Câu trả lời nào giải thích chính xác nhất tại sao tác giả chọn cụ thể middle depthwise layer để tăng kích thước kernel?



Hình 1: YOLOv10 CIB với middle 3×3 DW được enlarge thành 7×7 DW

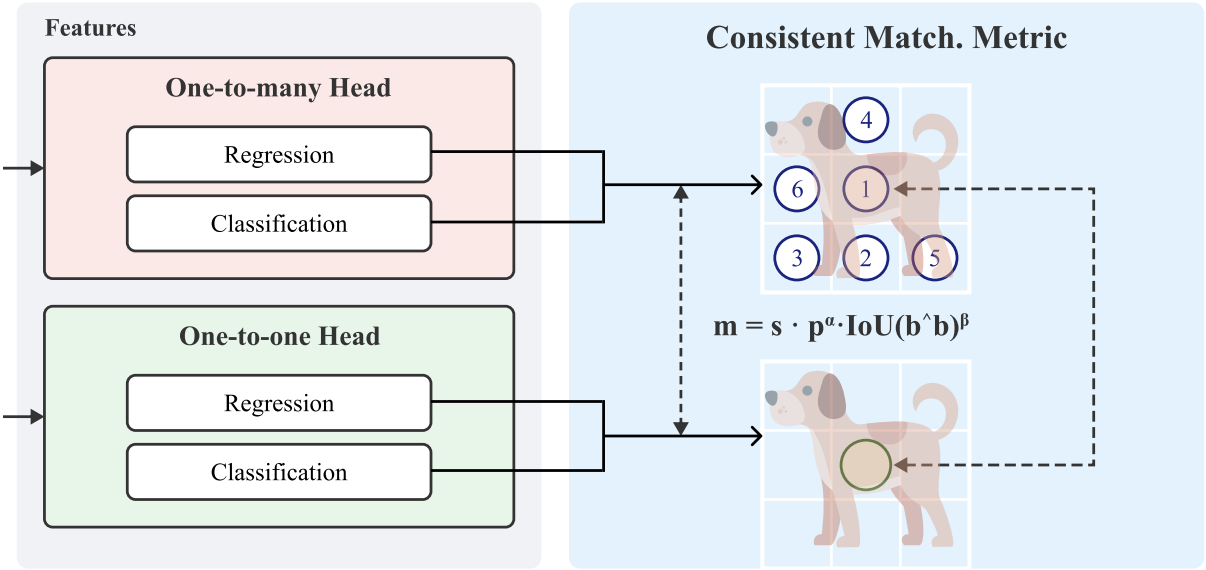
(a) Middle layer là vị trí tính toán hiệu quả nhất để áp dụng large kernel, giúp giảm thiểu số lượng tham số và chi phí tính toán FLOPs một cách tối ưu, trong khi vẫn duy trì hiệu năng cao cho việc phát hiện vật thể nhỏ trong các kịch bản phức tạp.

(b) Middle DW được áp dụng trên đặc trưng đã được mở rộng kênh bởi lớp 1×1, nhờ đó có thể tăng receptive field một cách hiệu quả về mặt tính toán. DW đầu tiên ưu tiên bảo toàn cấu trúc và chi tiết nhỏ, trong khi DW cuối đóng vai trò tinh chỉnh biểu diễn trước khi xuất ra khỏi block.

(c) Final DW cần giữ kernel 3×3 vì nếu mở rộng kernel ở tầng cuối cùng sẽ làm mờ (over-smooth) các đặc trưng không gian quan trọng tại đầu ra của block, từ đó làm giảm đáng kể độ chính xác định vị và khả năng phát hiện vật thể nhỏ trong ảnh.

(d) First DW nên được mở rộng thành 7×7 để thu thập thông tin không gian sớm và tốt hơn cho vật thể nhỏ, đồng thời cải thiện đáng kể quá trình trích xuất đặc trưng từ dữ liệu đầu vào ban đầu.

Câu 9: Xét một ảnh có **1 Ground Truth (GT)** duy nhất thuộc lớp `dog`. Trên feature map 3×3, mô hình tạo ra 6 candidate predictions (được đánh số $1 \to 6$) xung quanh GT như minh họa.



Ghi chú: Các candidate trong được đánh số $1 \to 6$ và **tương ứng đúng** với các dòng $i = 1 \to 6$ trong bảng dữ liệu bên dưới.
Để ghép (match) candidate với GT, YOLOv10 sử dụng **Consistent Matching Metric** cho **cả hai nhánh (heads)**. Công thức tổng quát là $m = s \cdot p^{\alpha} \cdot \text{IoU}^{\beta}$. Trong bài tập này, giả sử $\alpha = 1, \beta = 1$ để đơn giản hoá, ta có:

$$m_i = s_i \cdot p_i \cdot \text{IoU}_i.$$

Trong đó:

- $p_i = \Pr(\texttt{dog} \mid P_i)$ là xác suất phân lớp (từ **classification head**),
- $\text{IoU}_i = \text{IoU}(\hat{b}_i, b)$ là mức khớp bounding box (từ **regression head**),
- s_i là hệ số **spatial prior**. **Trong bài này,** s_i **được cho sẵn trong bảng** (không cần suy luận từ hình).

Các thông số (s_i, p_i, IoU_i) của 6 candidate được cho như sau:

i	s_i	p_i	IoU_i
1	1.0	0.80	0.70
2	1.0	0.60	0.90
3	0.5	0.95	0.60
4	0.5	0.70	0.80
5	0.2	0.90	0.50
6	0.2	0.40	0.90

Trong quá trình **Training**, nhánh **one-to-many** chọn **top-k** candidates với $k = 3$ theo điểm m_i. Trong pha **Inference**, mô hình chỉ sử dụng **nhánh one-to-one** (bỏ nhánh one-to-many), và giữ **top-1** candidate có m_i cao nhất để suy luận **NMS-free**.

Câu hỏi: Hãy tính m_i và chọn ra đúng tập candidates được match trong pha Training (top-3) và pha Inference (top-1).

(a) $m = \{0.560,\ 0.540,\ 0.285,\ 0.280,\ 0.090,\ 0.072\}$; Train top-3: $\{1, 2, 3\}$; Infer top-1: 1.

(b) $m = \{0.560,\ 0.540,\ 0.285,\ 0.280,\ 0.090,\ 0.072\}$; Train top-3: $\{1, 2, 4\}$; Infer top-1: 2.

(c) $m = \{0.560,\ 0.540,\ 0.285,\ 0.280,\ 0.090,\ 0.072\}$; Train top-3: $\{2, 3, 4\}$; Infer top-1: 2.

(d) $m = \{0.560,\ 0.540,\ 0.285,\ 0.280,\ 0.090,\ 0.072\}$; Train top-3: $\{1, 3, 4\}$; Infer top-1: 3.

Câu 10: Trong notebook (notebook): sau chúng ta sử dụng YOLOv10 để detect 3 bức ảnh tương ứng là: test_1 (khó thấy object), test_2 (đông người), test_3 (1 object là mèo).

```
# Load YOLOv10 (sẽ tự download yolov10n.pt nếu chưa có)
model = YOLO("yolov10n.pt")

out_dir = Path("runs/yolov10_demo")
out_dir.mkdir(parents=True, exist_ok=True)

imgsz = 640
conf = 0.25  # chỉ vẽ các box có conf >= 0.25

print("=== PREDICTIONS (bbox + conf) ===")
for name, p in [("test_1", p1), ("test_2", p2), ("test_3", p3)]:
    r = model.predict(source=str(p), imgsz=imgsz, conf=conf, verbose=
    False)[0]

    # Save annotated image
    save_path = out_dir / f"{p.stem}_pred_conf{conf}.jpg"
    r.save(filename=str(save_path))

    # Print summary
    num_boxes = int(len(r.boxes)) if (r.boxes is not None) else 0
    max_conf = float(r.boxes.conf.max().item()) if (r.boxes is not None
    and len(r.boxes) > 0) else 0.0
    print(f"{name}: num_boxes={num_boxes}, max_conf={max_conf:.2f} -> {
    save_path}")

    # Show annotated image
    display(Image(filename=str(save_path)))
```

Yêu cầu: Bạn hãy điền đoạn code trên vào vị trí #INSERT CODE HERE trong notebook. Quan sát kết quả in ra và cho biết label (class có max_conf lớn nhất) và max_conf của từng hình (test_1, test_2, test_3).

a) test_1: label=car, max_conf=0.73; test_2: label=person, max_conf=0.59; test_3: label=cat, max_conf=0.99

b) test_1: label=car, max_conf=0.75; test_2: label=person, max_conf=0.57; test_3: label=cat, max_conf=0.92

c) test_1: label=person, max_conf=0.75; test_2: label=car, max_conf=0.57; test_3: label=cat, max_conf=0.92

d) test_1: label=car, max_conf=0.75; test_2: label=person, max_conf=0.52; test_3: label=cat, max_conf=0.92